

Cyber Cipher

Anonymous Complaint Platform

Consolidated Design Baseline v1.0

Updated SRS, threat and privacy model, cryptographic protocol, canonical schemas and vectors, data/API contracts, architecture, roadmap, and security acceptance plan

Team Cyber Ciphers | 25 September 2026

Document set

1. Updated Systems Requirements Specification
2. Threat Model and Privacy/Data-Flow Model
3. Cryptographic Protocol Specification
4. Canonical CBOR Schemas and Test Vectors
5. Database and API Contracts
6. Service Architecture
7. Phased Implementation Roadmap
8. Security and Acceptance-Test Plan

Updated Systems Requirements Specification

1. Purpose and scope

Cyber Cipher allows a person whose eligibility has been verified against a national identity record to submit at most one anonymous complaint for a published matter. It supports encrypted evidence, device loss and replacement, anonymous two-way follow-up, independently verifiable case history, public redacted summaries, and anonymous community assessment.

The system does not determine whether an allegation is true, authenticate the real-world origin of evidence, guarantee anonymity against a global traffic-analysis adversary, or identify the author of an abusive anonymous complaint. Community votes are public accountability signals, not judicial or factual findings.

2. Security and privacy position

Eligibility is identified at enrollment, but complaint submission is unlinkable at the application-data layer. No component is permitted to store a mapping from a complaint to a NIC. The IdA learns enrollment and matter-entitlement issuance. The complaint domain learns a valid entitlement, a valid current membership proof, and an anonymous complaint. Separation, blind issuance, zero-knowledge proofs, minimized metadata, and optional Tor access jointly reduce linkability.

The system provides verifiability after acceptance. It cannot prove that an operator did not block a submission before issuing a signed receipt. A receipt proves acceptance; later log inclusion and consistency proofs prove that the accepted record was incorporated into the public append-only history.

3. Actors and trust boundaries

Actor	Permitted knowledge and authority	Prohibited knowledge or authority
Complainant	Own secrets, complaints, receipts, mailbox history	Other users' records
Identity Authority (IdA)	NIC record, active device record, issuance fact, recovery public key	Complaint content, receipt, entitlement value, complaint ID
Complaint Service (CS)	Public proof inputs, ciphertext, routing metadata, lifecycle state	NIC, phone number, person secret, plaintext evidence
Handler organization	Plaintext for assigned complaints; signed case actions	NIC or IdA enrollment data; unrelated complaints
Privacy reviewer	Proposed redacted derivative and minimum required context	Raw identity data; unrelated evidence
Transparency-log operator	Public commitments and event metadata	Complaint plaintext, attachment plaintext, NIC
Witness	Log tree data and signed tree heads	Complaint plaintext or identity data
Public auditor	Public log, checkpoints, redacted summaries, aggregate votes	Private complaint content and identity records
Community member	Public redacted summary and anonymous voting proof	Raw evidence and complainant identity
Infrastructure administrator	Service health and content-free telemetry	Decryption keys or a complaint-to-NIC mapping

The IdA, complaint domain, and public-audit/log domain MUST be separately administered. Production decryption and signing keys MUST be protected by KMS/HSM controls. Two-person approval is required for destructive key operations and emergency access.

4. Core functional requirements

4.1 Enrollment and active membership

- **FR-E1:** The client MUST generate a random non-zero BN254 person secret P and device secret D. P is persistent across legitimate device replacement; D is device-specific.
- **FR-E2:** D and other device-bound credentials MUST be sealed behind device unlock in hardware-backed storage where supported.
- **FR-E3:** The IdA MUST verify NIC eligibility and control of the registered contact channel. Production deployments MUST define the authoritative register integration and failure policy.
- **FR-E4:** One NIC MUST have at most one active device membership at a time.
- **FR-E5:** The active leaf MUST bind P and D using the domain-separated Poseidon construction defined in the cryptographic specification.

- **FR-E6:** The IdA MUST maintain a depth-16 append-only membership tree. Revocation replaces the leaf with `EMPTY_LEAF`; a revoked index MUST NOT be reused.
- **FR-E7:** Membership changes MUST be batched at most every 30 seconds. A new checkpoint is published only when the tree changed.
- **FR-E8:** The IdA MUST NOT expose an API that accepts a complaint identifier or complaint content.

4.2 Matters and blind entitlements

- **FR-T1:** A matter is identified by a random UUIDv4 plus an unsigned 32-bit version. Security-relevant changes and reopening MUST create a new version.
- **FR-T2:** Each matter version MUST use an independent RSA-3072 key pair and MUST publish the matter and public key at least 24 hours before submissions open.
- **FR-T3:** The IdA MUST issue at most one entitlement per NIC per matter version and store only the issuance fact.
- **FR-T4:** Issuance MUST use RSABSSA-SHA384-PSS-Randomized blind signing. The IdA MUST NOT learn the unblinded message or signature.
- **FR-T5:** The entitlement message MUST include the protocol version, full SHA-256 `matterKeyId`, random 128-bit serial, and a Poseidon commitment to `P` with fresh non-zero randomness `r`.
- **FR-T6:** An entitlement is valid only during the matter submission window. Its private key MUST be destroyed within 24 hours after the window closes. A reopened matter uses a new version and key.
- **FR-T7:** The CS MUST reject a repeated (`matterKeyId`, `serial`) and a repeated complaint nullifier. Serial consumption MUST occur only in the transaction that successfully accepts the submission.

4.3 Private membership and complaint authorization

- **FR-Z1:** The prototype MUST use a Groth16/BN254 proof generated by a Circom 2 circuit and verified against a content-addressed circuit artifact.
- **FR-Z2:** A complaint proof MUST show knowledge of `P`, `D`, `r`, and a current Merkle path such that the active leaf, entitlement person commitment, and matter-specific complaint nullifier all use the same `P`.
- **FR-Z3:** The complaint proof MUST bind the matter, entitlement serial, complaint commitment, and one-time submission challenge as public inputs.
- **FR-Z4:** The CS MUST accept only the current membership root carried by a valid Ed25519-signed proof-session lease with a maximum lifetime of 60 seconds.
- **FR-Z5:** The current prototype MUST expose a membership-provider interface so that a future custom zero-knowledge RSA accumulator can replace the Merkle provider without changing entitlement, receipt, or complaint APIs.

4.4 Complaint submission and evidence

- **FR-S1:** The client MUST normalize complaint text to UTF-8 NFC and create a deterministic-CBOR manifest.
- **FR-S2:** The public complaint commitment MUST be `SHA-256(random 32-byte salt || deterministic-CBOR manifest)`.
- **FR-S3:** Client preprocessing MUST remove metadata or create a safe derivative before the committed public form is calculated. Encrypted originals MAY be retained for the handler and MUST have separately recorded hashes.
- **FR-S4:** The prototype allowlist is PDF, JPEG, PNG, MP4, M4A, and plain text; the limits are 25 MB per file and 100 MB per complaint. Archives, executables, scripts, and active document content MUST be rejected.
- **FR-S5:** Each complaint package MUST be encrypted with a random AES-256-GCM data-encryption key and 96-bit nonce. The data key MUST be wrapped to the assigned handler organization's versioned key using HPKE X25519/HKDF-SHA256/AES-256-GCM.
- **FR-S6:** The CS MUST verify the RSA entitlement and ZK proof before accepting the submission. Verification and insertion of complaint, serial-spend, nullifier, initial log event, and receipt record MUST be atomic.
- **FR-S7:** The CS MUST return a randomized deterministic-CBOR receipt signed by a dedicated Ed25519 receipt key.
- **FR-S8:** Submission error responses MUST not distinguish unknown, previously spent, expired, or malformed entitlements to an unauthenticated caller.
- **FR-S9:** The service MUST support direct TLS and a Tor-compatible endpoint. It MUST NOT persist source IP, TLS fingerprint, advertising identifier, or third-party analytics identifier with complaint data.

4.5 Recovery and device replacement

- **FR-R1:** Enrollment MUST create a random 128-bit recovery seed and random 128-bit recovery ID. The recovery seed MUST remain under user control.
- **FR-R2:** The recovery authentication key MUST be an Ed25519 key deterministically derived using HKDF-SHA256 with a domain-specific label. The IdA stores only its public key and recovery record.
- **FR-R3:** The encrypted person-secret backup MUST use AES-256-GCM with a key derived from the recovery seed, the recovery ID as salt, and a fixed protocol label.
- **FR-R4:** Recovery requires a fresh IdA challenge valid for five minutes and a signature from the recovery key over the challenge and new device public material.
- **FR-R5:** Successful recovery MUST atomically revoke the old leaf, allocate a never-before-used leaf index for the new device, preserve P, rotate D, rotate the recovery ID/seed/key, and publish the membership change.
- **FR-R6:** Failed recovery MUST NOT change active membership. Recovery secrets MUST never be requested through a service-line operator.

4.6 Anonymous follow-up

- **FR-F1:** Every complaint MUST use a fresh random 256-bit followUpSeed, random 128-bit mailboxId, derived Ed25519 authentication key, and derived X25519 encryption key.
- **FR-F2:** The CS stores only the mailbox ID and public keys. Mailbox authentication uses signed server challenges; replies use HPKE.
- **FR-F3:** Mailbox secrets MUST NOT be derived from the entitlement serial, P, D, NIC, or recovery ID.
- **FR-F4:** Mailbox seeds MAY be placed in a user-controlled AES-256-GCM recovery bundle encrypted from the recovery seed. The IdA and CS MUST NOT receive plaintext bundle contents.
- **FR-F5:** After successful device recovery, the client MUST re-encrypt the user-controlled bundle under the rotated recovery credential.

4.7 Handling and reassignment

- **FR-H1:** Each handler organization MUST have a versioned X25519 HPKE key pair protected by its KMS/HSM.
- **FR-H2:** Staff access MUST use phishing-resistant WebAuthn and least-privilege role assignments.
- **FR-H3:** The handler portal MUST decrypt only assigned cases. Every read, state change, export, and key rewrap MUST create an attributable signed audit event.
- **FR-H4:** Internal reassignment within the same organization uses authorization changes. Cross-organization reassignment requires the current handler to rewrap only the data key to the new organization's public key and create a signed transparency-log event.
- **FR-H5:** The prototype MUST NOT contain a universal complaint-decryption or escrow key.
- **FR-H6:** The lifecycle is SUBMITTED, ACKNOWLEDGED, ASSIGNED, UNDER_REVIEW, optionally ACTION_REQUIRED, then RESOLVED, REJECTED, or CLOSED. Public visibility states such as SUPPRESSED do not erase the private case.
- **FR-H7:** Default service targets are acknowledgment within 24 hours, assignment within 72 hours, and initial finding within 14 days. Signed extensions and reassignment reasons MUST be logged. Appeals remain open for 14 days after a final finding.

4.8 Redaction, community voting, and auditor comparison

- **FR-C1:** Public release is opt-in. The handler creates a redacted derivative, automated PII checks run, and an independent privacy reviewer approves it. Raw evidence MUST NOT be public.
- **FR-C2:** Community voting uses a separate Semaphore-style circuit with one nullifier per person per complaint.
- **FR-C3:** Voting uses the current root and a signed 60-second lease. Choices are CORROBORATE, DISPUTE, INSUFFICIENT_INFORMATION, and UNSAFE_OR_ABUSIVE.
- **FR-C4:** The default window is seven days. A published outcome requires at least 10 valid voters and at least 60% support for a principal outcome; otherwise it is INCONCLUSIVE.
- **FR-C5:** A vote is an assessment of the redacted information, not verification of truth.
- **FR-C6:** Comparison states are ALIGNED, CONFLICTING, COMMUNITY_INCONCLUSIVE, and HANDLER_PENDING. Conflict triggers auditor review, not automatic deletion or case closure.

- **FR-C7:** An auditor MAY freeze a public listing, request handler justification, or escalate under policy. Every action and reason MUST be logged. Only an authorized case authority may close the underlying complaint.

4.9 Transparency and receipts

- **FR-V1:** The log MUST be an RFC 6962-style append-only SHA-256 Merkle tree with domain-separated leaf and node hashes.
- **FR-V2:** Signed tree heads MUST be issued when the log changes, with a target interval of 30 seconds.
- **FR-V3:** Three administratively independent witnesses MUST verify the tree. Two matching witness signatures are required for a checkpoint to be final.
- **FR-V4:** The log MUST provide inclusion and consistency proofs and an unauthenticated bulk-download/auditor interface.
- **FR-V5:** Public entries MUST contain commitments and minimum event metadata only; never NIC, plaintext, attachment, entitlement serial, person commitment, or ZK witness.
- **FR-V6:** The receipt MUST include protocol version, random 128-bit receipt ID, random 256-bit receipt nonce, complaint ID, matter ID/version, complaint commitment, acceptance time, log-entry hash, and receipt-key ID.
- **FR-V7:** The receipt MUST exclude entitlement serial, membership root, person commitment, and nullifier. It MUST be deterministic-CBOR encoded and signed with Ed25519.
- **FR-V8:** The client MUST verify the receipt signature and later verify log inclusion. A receipt without final inclusion after the published service objective MUST display a clear warning.

5. Non-functional requirements

ID	Requirement
NFR-1	Monthly intake availability target: 99.5% for the prototype. Accepted complaints MUST survive loss of one storage node.
NFR-2	A proof-only submission excluding large-file upload SHOULD be acknowledged within three seconds at p95 under declared prototype load.
NFR-3	Receipt and inclusion verification SHOULD finish within one second on a representative mid-range phone, excluding network retrieval.
NFR-4	The depth-16 tree supports 65,536 issued membership indices. Indices are never reused; capacity monitoring MUST alert at 80%.
NFR-5	The UI MUST support Sinhala, Tamil, and English, screen readers, keyboard navigation, and WCAG 2.2 AA contrast and focus behavior.
NFR-6	Verification MUST be reachable within two user actions from a complaint and explain success/failure in plain language.
NFR-7	Prototype operational logs are retained for 30 days without complaint content or raw IP. Encrypted evidence defaults to 12 months after closure unless legal hold applies. Public commitments remain permanent.
NFR-8	Cryptographic artifacts and application builds MUST be reproducible. Artifact IDs are full SHA-256 hashes of deterministic manifests.
NFR-9	All clocks MUST use authenticated time synchronization and tolerate an explicitly configured maximum skew.
NFR-10	Rate limits and abuse defenses MUST not create a persistent identifier joinable to a complaint.

6. Explicit limitations

- A stolen unlocked phone can act until the next membership update takes effect.
- A compromised device can reveal locally held receipts, entitlement values, and mailbox secrets.
- Optional Tor reduces network linkability; direct TLS does not hide the user's address from the ingress network.
- A global observer may correlate timing despite batching and minimized logs.
- Signed receipts improve integrity but weaken receipt-freeness under coercion.
- Community consensus cannot establish factual truth.
- A malicious client may submit manipulated evidence; the platform proves bytes and history, not real-world authenticity.
- If IdA and CS collude and also possess sufficiently precise external traffic/device telemetry, application-layer unlinkability does not eliminate all correlation risk.

7. Retention defaults

- Expired challenges and leases: deletion within 24 hours.
- Content-free operational security logs: 30 days.
- Encrypted complaint evidence: 12 months after closure, unless jurisdiction or legal hold requires otherwise.
- Nullifiers and spent serials: matter lifetime plus audit period.

- Redacted public entries, commitments, signed checkpoints, witness signatures, and consistency history: permanent.
- RSA matter private keys: destruction within 24 hours after submission close.

All production retention periods require legal and data-protection approval in the deployment jurisdiction.

8. Acceptance boundary

The prototype is ready for controlled testing when all acceptance tests in 08_SECURITY_AND_ACCEPTANCE_TEST_PLAN.md pass. It is not ready for real complainants until an external cryptographic review, penetration test, privacy review, accessibility test, operational recovery exercise, and jurisdictional legal review are complete.

9. Correction register against the supplied documents

Source statement/problem	Corrected baseline
The RSA accumulator description did not define a complete private-membership proof or bind membership to the blind entitlement.	Prototype uses an explicit Semaphore-style Merkle proof. The same hidden P determines active membership, entitlement person commitment, and matter nullifier. A custom RSA accumulator remains a future provider.
The fallback proposed a Merkle non-membership proof over revoked devices.	The prototype proves membership in the current active-member tree; revoked leaves are replaced with a fixed empty value.
Blind RSA used an informal full-domain hash and "strong-prime" modulus.	Use the RFC 9474 RSABSSA-SHA384-PSS-Randomized suite, independent FIPS 186-5-compatible RSA-3072 matter keys, and exponent 65537.
Entitlement/device revocation semantics were underspecified.	Entitlement is bound to persistent P; the ZK proof also requires current device D. Device replacement rotates D while preserving P, so unspent entitlements remain usable only by the recovered active membership.
Phone loss relied on a service-line/hotline report.	Recovery uses a user-held 128-bit seed, derived Ed25519 recovery key, signed five-minute challenge, and mandatory atomic credential rotation.
Follow-up key was derived from the public entitlement serial.	Each complaint gets an independent random 256-bit follow-up seed and separate mailbox authentication/encryption keys.
Complaint content was visible to the complaint service.	Client encrypts the package with AES-256-GCM and HPKE-wraps the data key to the assigned handler organization. The CS stores ciphertext only.
Receipt format and privacy fields were not fixed.	Random receipt ID/nonce, deterministic CBOR, dedicated Ed25519 signature, and an explicit exclusion list for serial/nullifier/root/person commitment.
Log wording implied that deleting a private database row necessarily breaks a log consistency proof.	The log proves its own append-only public history. A receipt and included leaf prove prior acceptance; missing private data is exposed as retrieval/durability failure, not as a false consistency-proof claim.
Network-observer claim implied Tor and batching prevented timing linkage.	Tor is optional and reduces linkability; direct ingress and global timing correlation remain explicit residual risks.
Community visibility relied on account age/weighted accounts, conflicting with unlinkable membership.	Anonymous one-person-per-complaint vote proofs, fixed choices, seven-day window, quorum 10, 60% decision threshold, and an auditor-controlled discrepancy workflow.
Community voting could be read as verification of truth or automatic shutdown authority.	Votes assess redacted public information only. Conflict triggers review; only an authorized case authority closes the private complaint.
Hardware-backed Ed25519 availability was assumed uniformly.	The protocol separates BN254 device membership secret from platform authentication and requires hardware-backed sealing where supported; staff authentication uses WebAuthn.
Matter/key identity, setup artifacts, and root leases were not fully versioned.	UUIDv4 plus uint32 matter version, full SPKI fingerprints, SHA-256 content-addressed circuit artifacts, signed checkpoints, and 60-second current-root leases are mandatory.

Threat Model and Privacy/Data-Flow Model

1. Method and protected assets

This threat model uses asset/trust-boundary analysis with STRIDE-style categories and privacy-linkability analysis. It treats service operators, administrators, handlers, and network observers as possible adversaries. The most sensitive assets are:

- the hidden relationship between a NIC and a complaint;
- person secret P, device secret D, recovery seed, mailbox seed, and entitlement token;
- plaintext complaint and evidence;
- handler and witness private keys;
- integrity of membership roots, matter keys, lifecycle history, votes, receipts, and log checkpoints;
- availability of enrollment, recovery, submission, and audit verification.

2. Trust zones

Diagram source is preserved in the editable Markdown package.

No private network route, database credential, administrator role, telemetry stream, or backup account may span the identity and complaint zones.

3. Privacy data inventory

Data	Created/seen by	Stored by	Must not be joined with
NIC and registered contact	IdA	IdA enrollment store	Complaint ID, receipt ID, mailbox ID, entitlement serial
P	Client	Device and user recovery bundle	NIC outside the user's device
D	Client	Hardware-backed device storage	Complaint record
Membership leaf/index	Client, IdA	IdA	Complaint ID or entitlement serial
Issuance fact	IdA	IdA	Blind message or unblinded token
Entitlement serial/signature	Client, CS at spend	Client; CS spent-token store	NIC or enrollment record
Complaint nullifier	Client, CS	CS	NIC, P, membership index
Complaint ciphertext	Client, CS, handler	CS/object store	IdA data
Complaint plaintext	Client, assigned handler	Handler memory/authorized workspace	IdA data and public log
Receipt	Client, CS	Client; minimal CS receipt record	NIC, serial, nullifier, root
Mailbox public keys	Client, CS, handler	CS	NIC and entitlement serial
Mailbox private seed	Client	User-controlled encrypted bundle	Any server-side record in plaintext
Public vote	Community client, portal	Community portal/log	Member identity or membership index
Network metadata	Ingress transiently	Short-lived isolated counters	Complaint database

4. Principal adversaries and controls

Adversary	Capability	Required outcome	Primary controls	Residual risk
Malicious IdA	Reads/changes enrollment and issuance data	Cannot read complaints or recognize a spent blind token	Blind RSA issuance, no complaint API, separate zone	Timing and external telemetry correlation
Malicious CS	Reads/changes complaint DB and API	Cannot learn NIC or plaintext evidence; modifications detectable	Client encryption, ZK membership, receipts, transparency log	Can refuse service before receipt; traffic observation at ingress
IdA and CS collusion	Combines application databases	No deterministic application-data link from complaint to NIC	Blinding, ZK, distinct random values, data minimization	Correlation remains possible using precise timing/device/network side channels
Malicious handler	Legitimate access to one case	Cannot browse unrelated cases or identify complainant	Organization-scoped encryption, RBAC/ABAC, WebAuthn, signed access log	Authorized handler can leak plaintext they legitimately see
Log operator	Equivocates or rewrites log	Divergence detected	RFC 6962 proofs, three witnesses, 2-of-3 finality, gossip	Two witnesses colluding with operator can finalize a split view
Witness compromise	Signs false tree	One compromised witness insufficient	2-of-3 quorum and independent administration	Two compromised witnesses defeat quorum assumption
Phone thief	Controls an unlocked device	Access ends after root update; recovery replaces device	Device unlock, current-root proofs, 30-second updates, recovery rotation	Actions before revocation can be valid and cannot always be undone
Recovery thief	Obtains recovery seed	Cannot silently retain access after successful owner recovery	Five-minute challenge, atomic rotation, recovery event logging	Seed theft before rotation enables takeover
Network observer	Sees traffic timing/address	Direct complaint-to-person inference is reduced	Optional Tor, batching, no IP persistence, padding policy	Global correlation and device compromise are out of scope
Sybil attacker	Creates accounts/devices	One eligible person contributes once per scope	NIC enrollment, one active membership, ZK nullifiers	Registry duplicates/ghost identities transfer into system
Community brigade	Coordinates valid voters	Cannot cast multiple votes per person; result not treated as fact	Vote nullifier, quorum, threshold, auditor comparison	Many legitimately eligible coordinated voters can influence signal
Malicious uploader	Sends parser exploits/malware	Handler environment remains contained	Allowlist, size limits, quarantine, isolated conversion/scanning	Novel parser exploits; encrypted content prevents CS-side inspection
Coercer	Forces user to reveal receipt	Platform cannot confirm identity mapping	No mapping exists; export warning	A user-held signed receipt can be coerced from the user
Availability attacker	Floods APIs or storage	Accepted complaints not lost; anonymous users retain access	Queueing, bounded parsing, rate controls, replicas	Strong anonymous rate limits are difficult without linkability

5. Corrected security claims

The system MAY claim:

- no intentional application database contains a NIC-to-complaint mapping;
- a valid proof establishes active membership without disclosing the membership leaf or index;
- a blind entitlement cannot be recognized by the IdA from the final token under the protocol assumptions;
- accepted complaints and later events are tamper-evident after witnessed log inclusion;
- one proof nullifier per person and scope is enforced assuming circuit correctness and P secrecy.

The system MUST NOT claim:

- information-theoretic or global-observer anonymity;
- that Tor alone prevents all timing correlation;
- that the log proves a request was never blocked before acceptance;
- that community voting verifies factual truth;
- that commitments prove evidence authenticity;
- that an RSA accumulator is deployed in the prototype;
- that compromise of the user's unlocked device preserves their local privacy.

6. Data flows

6.1 Enrollment

1. The client generates P, D, recovery seed/ID, and recovery public key.
2. NIC and registered-channel evidence travel only to the IdA.
3. The IdA validates eligibility, allocates a new leaf index, and stores the leaf/public recovery material.
4. The IdA batches the membership change and publishes a signed checkpoint.
5. The client receives its index/path data and stores secrets locally.

Privacy rule: complaint services do not participate in this flow.

6.2 Blind entitlement issuance

1. The client creates a random serial and $\text{personCommitment} = \text{Poseidon}(\text{domain}, P, r)$.
2. It encodes the entitlement message, applies randomized preparation, and blinds the RSA representative.
3. The IdA verifies that this NIC has no issuance record for the matter version.
4. The IdA atomically records the issuance fact and signs the blinded value.
5. The client unblinds and verifies the entitlement.

Privacy rule: the IdA never receives the final encoded message, serial, commitment, or signature.

6.3 Submission

1. The client sanitizes public derivatives, builds the manifest, and calculates the salted commitment.
2. It encrypts the complaint package with AES-256-GCM and HPKE-wraps the data key to the handler organization.
3. It obtains a signed challenge/root lease and generates the Groth16 proof against that root.
4. Through Tor or direct TLS, it sends entitlement, proof envelope, ciphertext, wrapped key, mailbox public data, and public commitment.
5. The CS verifies matter/window/key, challenge, lease, RSA signature, proof artifact, public inputs, unused serial, and unused nullifier.
6. In one transaction, the CS consumes the serial/nullifier, stores ciphertext, appends the initial event, and creates the signed receipt.
7. The client later obtains and verifies inclusion in a 2-of-3 witnessed checkpoint.

Privacy rule: the receipt excludes serial, nullifier, membership root, and person commitment.

6.4 Device-loss recovery

1. A replacement device obtains a five-minute challenge using the random recovery ID.
2. The user supplies the recovery seed locally; the device derives the recovery signing key and decrypts the local/user-controlled backup of P.
3. The replacement device generates new D and recovery credentials, then signs the challenge and new public material.
4. The IdA atomically marks the old leaf empty, allocates a fresh index, stores the new recovery public key, and invalidates the old recovery record.
5. The next membership checkpoint makes old-device proofs invalid.

Privacy rule: the recovery request contains no complaint, entitlement, receipt, or mailbox identifier.

6.5 Follow-up

1. The client authenticates to its random mailbox by signing a one-time challenge with the mailbox Ed25519 key.
2. Handler replies are HPKE-encrypted to the mailbox X25519 key.
3. Client replies are encrypted to the handler organization and signed by the mailbox authentication key.
4. The CS routes ciphertext but has no mailbox private key.

6.6 Public redaction and voting

1. The handler creates a redacted derivative; automated PII checks and an independent reviewer approve it.
2. The portal publishes the derivative, its commitment, provenance, and consent state.
3. An active member obtains a current-root lease, produces a vote proof and nullifier, and submits one of four choices.
4. The portal publishes aggregate results only after the seven-day window and quorum rules.
5. The auditor compares the community assessment with handler findings and logs any escalation or public freeze.

7. Linkability analysis

Pair	Intended relationship	Mitigation
Enrollment <-> complaint	Unlinkable	Blind token, ZK membership, no shared IDs, separate zones
Two complaints by same person for different matters	Unlinkable	Matter-scoped nullifiers and fresh serials/commitments
Complaint <-> vote by same person	Unlinkable	Different domain/nullifier scopes and separate circuits
Complaint <-> its follow-up mailbox	Intentionally linkable inside the case	Random mailbox ID; no identity or cross-case key reuse
Old device <-> replacement device	Linkable to IdA	Required for revocation; must not leave identity zone
Public summary <-> private complaint	Intentionally verifiable	Explicit derivative commitment/provenance; no raw evidence

8. Abuse and incident rules

- Do not log proof witnesses, decrypted payloads, recovery seeds, or cryptographic private values.
- Authentication failures exposed to unauthenticated callers use uniform error categories and bounded timing.
- A suspected circuit bug, matter-key compromise, membership-checkpoint signing compromise, or two-witness inconsistency is a stop-issuance/stop-acceptance incident for the affected scope.
- A handler encryption-key compromise pauses new assignment to that key, publishes a signed key-status event, rotates the key, and rewraps affected data keys where possible.
- A public-content safety issue may freeze visibility but must not erase the private case or transparency history.
- Security telemetry must be content-free and must not create a durable identifier joinable to complaints.

9. Privacy verification checklist

- Database snapshots from IdA and CS contain no common stable identifier capable of deterministic joining.
- Ingress logs cannot be joined to complaint rows by exact timestamp; timestamp precision and retention are minimized.
- Analytics, crash reporting, push notifications, and CDN logs are disabled or privacy-reviewed before use.
- Error reporting redacts CBOR payloads, proofs, ciphertext metadata, and mailbox identifiers.
- Backups preserve zone separation and use independent credentials and encryption keys.
- Public log schemas are allowlisted; unknown fields are rejected rather than accidentally published.

Cryptographic Protocol Specification

1. Version and notation

This specification defines protocol version 1.

- $H(x)$ is SHA-256 unless explicitly stated otherwise.
- $H384(x)$ is SHA-384.
- $HKDF(salt, IKM, info, L)$ is HKDF-SHA256.
- $CBOR(x)$ is RFC 8949 deterministic CBOR according to schemas/cyber-cipher-v1.cddl.
- $||$ is byte concatenation.
- $I2OSP(x, L)$ is an unsigned big-endian integer encoded in exactly L bytes.
- $q = 21888242871839275222246405745257275088548364400416034343698204186575808495617$, the BN254 scalar-field modulus.
- Field elements in external encodings are exactly 32-byte unsigned big-endian values strictly less than q .
- Random byte strings are produced by the operating system CSPRNG.

Every signed object uses an explicit ASCII signing prefix ending in NUL:

```
CYBER-CIPHER/v1/<object-type>\x00 || CBOR(unsigned-object)
```

Ed25519 signs these bytes directly. Implementations MUST NOT silently prehash them or substitute Ed25519ph.

2. Domain separation

For Poseidon, each domain constant is:

```
Domain(label) = OS2IP(SHA-256("CYBER-CIPHER/v1/poseidon/" || label)) mod q
```

The result MUST be non-zero. The protocol labels are:

```
person
device
member-leaf
empty-leaf
merkle-node
entitlement-person
complaint-nullifier
vote-nullifier
vote-message
matter-field
serial-field
complaint-id-field
commitment-field
challenge-field
```

Circuit source MUST contain the precomputed constants. The artifact manifest MUST bind those source files.

For non-circuit hashes, the input is length-framed:

```
Frame(label, parts...) =
  "CYBER-CIPHER/v1/" || label || 0x00 ||
  I2OSP(len(part1), 8) || part1 || ...
```

```
HashToField(label, parts...) = OS2IP(SHA-256(Frame(label, parts...))) mod q
```

3. Key catalogue

Key	Algorithm	Scope	Custody and rotation
Matter entitlement key	RSA-3072 RSASSA-PSS/SHA-384	One matter version	Prototype: encrypted PKCS#8 in dedicated secrets volume; production: HSM. Destroy private key within 24 hours after submission close.
Membership checkpoint key	Ed25519	IdA membership service	Dedicated KMS/HSM key; publish versioned public key before use.
Proof-session lease key	Ed25519	Complaint verifier	Dedicated key; separate from every other signer.
Recovery challenge key	Ed25519	IdA recovery service	Dedicated key.
Receipt key	Ed25519	Complaint service	Dedicated key; overlap public keys during rotation.
Transparency-log key	Ed25519	Log operator	Dedicated key.
Witness keys	Ed25519	One per witness	Each independently administered.
Handler envelope key	X25519 HPKE	One handler organization and key version	KMS/HSM; retain old private versions until all DEKs are rewrapped or data expires.
Handler action key	WebAuthn authenticator plus organization signing service	Named staff action	Phishing-resistant user authentication; action signature issued by controlled service.
Recovery key	Ed25519	One recovery generation	Derived locally; public key stored by IdA; rotate after use.
Mailbox auth key	Ed25519	One complaint	Derived locally from follow-up seed.
Mailbox encryption key	X25519	One complaint	Derived locally from follow-up seed.

An Ed25519/X25519 key ID is the full SHA-256 hash of the canonical DER SubjectPublicKeyInfo. `matterKeyId` is the full SHA-256 hash of the canonical DER RSA SubjectPublicKeyInfo, including the RSASSA-PSS parameters. Text display uses `base64url` without padding; protocol encodings use the raw 32 bytes.

4. RSA blind-signed entitlement

4.1 Key generation and publication

Generate a fresh RSA-3072 key with public exponent 65537 using a FIPS 186-5-compatible generator. Do not impose a custom "strong prime" construction. The key **MUST** be dedicated to RSABSSA for one matter version and **MUST NOT** be used for ordinary signatures, TLS, encryption, or another matter.

The public key metadata fixes:

- RSABSSA suite: RSABSSA-SHA384-PSS-Randomized;
- MGF: MGF1-SHA384;
- PSS salt length: 48 bytes;
- randomized message-preparation prefix: 32 bytes;
- modulus size: 3072 bits;
- public exponent: 65537.

Publish the matter, public key, fingerprint, opening/closing times, and circuit artifact IDs at least 24 hours before issuance/submission opens.

4.2 Entitlement message

The application message is deterministic CBOR containing:

```
protocolVersion = 1
matterKeyId     = 32 bytes
serial         = 16 random bytes
personCommitment = Field(Poseidon(Domain("entitlement-person"), P, r))
```

where P and r are random non-zero field elements. The RSABSSA randomized preparation creates a 32-byte `messageRandomizer`; the client **MUST** retain it with the final signature because verification reconstructs the prepared message.

4.3 Issuance

1. The client verifies matter metadata and key fingerprint.
2. It constructs and validates the entitlement message.
3. It performs RFC 9474 randomized preparation and blinding using fresh randomness.
4. Over an authenticated enrollment session, it sends the blinded request and matter version to the IdA.
5. The IdA checks the submission window/issuance policy and atomically inserts the unique issuance fact (NIC-internal-id, matter-id, matter-version).
6. The IdA blind-signs and returns the blind signature.
7. The client finalizes/unblinds and verifies locally before saving the token.

The IdA MUST NOT log blinded representatives at a precision/retention that enables later operational correlation. Failed issuance MUST not create the issuance fact unless the signing operation was completed and the response is recoverable idempotently.

4.4 Spend

The CS verifies the signature over the exact CBOR entitlement message using the published matter key and retained message randomizer. It checks matterKeyId, matter window, serial length, and scalar canonicity. RSA verification occurs outside the Groth16 circuit. The CS then checks that the proof public signals contain matching serialField and personCommitment.

5. Active-membership tree

5.1 Leaves

At enrollment:

```
personHash = Poseidon(Domain("person"), P)
deviceHash = Poseidon(Domain("device"), D)
leaf = Poseidon(Domain("member-leaf"), personHash, deviceHash)
EMPTY_LEAF = Poseidon(Domain("empty-leaf"), 0)
```

The client sends personHash and deviceHash to the IdA inside the identified enrollment channel. The IdA may associate these hashes and the allocated index with the NIC, but they MUST never appear in the complaint domain or public log. This stored person anchor permits recovery to preserve P without revealing it.

The depth is 16. Unallocated and revoked positions contain EMPTY_LEAF. Allocation is monotonic; an index is never reused. Internal nodes are:

```
node = Poseidon(Domain("merkle-node"), left, right)
```

5.2 Checkpoint

Every changed 30-second batch increments the unsigned 64-bit epoch by one. The membership service signs a deterministic-CBOR checkpoint containing protocol version, epoch, root, previous checkpoint hash, publication time, update-batch hash, and signing-key ID. The checkpoint hash is SHA-256 of its complete signed CBOR object and is appended to the transparency log.

Clients MUST verify the signature and predecessor hash before accepting a new path/root. A checkpoint without a valid predecessor is a fork warning.

6. Complaint proof

6.1 Private witness

- P: non-zero person scalar.
- D: non-zero current device scalar.
- r: non-zero entitlement-commitment randomness.
- merkleSiblings[16].
- merklePathBits[16], each constrained boolean.

6.2 Public signals, in exact order

1. membershipRoot
2. epoch
3. matterField
4. serialField
5. personCommitment
6. complaintNullifier
7. complaintCommitmentField
8. challengeField

The derivations are:

```
matterField = HashToField("matter-field", matterId16, uint32be(matterVersion))
serialField = HashToField("serial-field", serial16)
personCommitment = Poseidon(Domain("entitlement-person"), P, r)
complaintNullifier = Poseidon(Domain("complaint-nullifier"), P, matterField)
complaintCommitmentField = HashToField("commitment-field", complaintCommitment32)
challengeField = HashToField("challenge-field", SHA-256(CBOR(unsignedLease)))
```

The circuit recomputes personHash, deviceHash, leaf, and the 16-level path; constrains the result to membershipRoot; recomputes the person commitment and nullifier; and exposes all public signals above. matterField, serialField, complaint commitment, and challenge are public binding signals even where no private arithmetic depends on them. The verifier MUST compare them to the request; omitting that comparison is a security failure.

6.3 External verification order

1. Parse with strict size/canonicity limits.
2. Resolve and verify artifactId and verification key.
3. Verify matter metadata and submission window.
4. Verify signed root lease and challenge; require current root/epoch and unexpired 60-second lease.
5. Verify RSA entitlement and derive expected public signals.
6. Verify Groth16 proof.
7. Check unique serial and complaint nullifier.
8. Validate ciphertext, HPKE envelope, mailbox public data, and commitment sizes.
9. Atomically accept, consume, log, and issue receipt.

The challenge ID is random 16 bytes. A challenge is consumed only by a successfully committed submission. Failed verification is rate limited but does not consume it. A successful transaction marks it consumed.

7. Voting proof

The voting circuit is separate from the complaint circuit.

Private witness: P, D, merkleSiblings[16], and merklePathBits[16].

Public signals, in exact order:

1. membershipRoot
2. epoch
3. complaintIdField
4. voteNullifier
5. voteChoice
6. voteMessageCommitment
7. challengeField

Derivations:

```
complaintIdField = HashToField("complaint-id-field", complaintId16)
voteNullifier = Poseidon(Domain("vote-nullifier"), P, complaintIdField)
voteChoice in {0,1,2,3}
voteMessageCommitment = Poseidon(Domain("vote-message"), complaintIdField, voteChoice)
```

Choice mapping: 0=CORROBORATE, 1=DISPUTE, 2=INSUFFICIENT_INFORMATION, 3=UNSAFE_OR_ABUSIVE.

The same current-root and 60-second signed-lease rules apply. The vote service atomically inserts the unique (complaintId, voteNullifier) and aggregate contribution only after proof verification.

8. Groth16 artifact lifecycle

- Proof system: Groth16.
- Curve: BN254.
- Circuit language: Circom 2.
- Prover target: Rapidsnark-compatible WASM/native artifacts.
- Phase 1: an existing, publicly verifiable Powers-of-Tau transcript at adequate constraint capacity.
- Phase 2: one circuit-specific ceremony per final circuit with three independent contributions.
- Toxic intermediate files MUST be destroyed by contributors after contribution.

The deterministic artifact manifest records protocol/circuit name and version, tree depth, proof system, curve, compiler and dependency versions, and SHA-256 hashes of source bundle, R1CS, WASM, proving key, verification key, Powers-of-Tau input, phase-two transcript, and creation timestamp. artifactId is the full SHA-256 hash of CBOR(manifest). A proof envelope includes scheme, circuit version, artifact ID, proof, and ordered public signals.

Any circuit-source, dependency, compiler, parameter, constraint, or public-input-order change creates a new artifact ID. Verifiers use an explicit allowlist; they MUST NOT fetch an arbitrary verification key named by an untrusted request.

9. Recovery protocol

9.1 Derivation

The 16-byte recovery seed has 128 bits of effective security. Derive:

```
recoverySignSeed = HKDF(recoveryId16, recoverySeed16,
  "CYBER-CIPHER/v1/recovery-sign", 32)
backupKey = HKDF(recoveryId16, recoverySeed16,
  "CYBER-CIPHER/v1/person-backup", 32)
bundleKey = HKDF(recoveryId16, recoverySeed16,
  "CYBER-CIPHER/v1/mailbox-bundle", 32)
```

recoverySignSeed is the Ed25519 private seed. The person backup and mailbox bundle use AES-256-GCM with fresh random 12-byte nonces and AAD containing protocol version, recovery ID, purpose, and generation.

9.2 Challenge and atomic rotation

The IdA returns a signed challenge with random 16-byte challenge ID, recovery ID, issue/expiry times, and server nonce. Maximum lifetime is five minutes. The replacement device signs the challenge hash, new deviceHash, new recovery public key/ID, and the stored personHash using the old recovery key.

Within one serializable transaction the IdA:

1. locks the recovery and enrollment rows;
2. verifies challenge freshness, non-consumption, signature, and personHash equality;
3. replaces the old leaf with EMPTY_LEAF;
4. allocates a new monotonic index and leaf using the same personHash and new deviceHash;
5. invalidates the old recovery credential and stores the new one;
6. consumes the challenge and records the update batch.

If any step fails, none are committed. The user re-encrypts local recovery bundles under the new seed after success.

10. Complaint encryption and handler reassignment

Generate random DEK (32 bytes) and nonce (12 bytes). Encrypt the deterministic-CBOR private complaint package using AES-256-GCM. AAD binds protocol version, complaint ID, matter ID/version, complaint commitment, and handler key ID.

Wrap DEK with RFC 9180 HPKE using:

- KEM: DHKEM(X25519, HKDF-SHA256);
- KDF: HKDF-SHA256;
- AEAD: AES-256-GCM;
- mode: base mode;
- info: "CYBER-CIPHER/v1/handler-dek" || matterId || version;
- AAD: complaint ID, complaint commitment, and handler key ID.

The CS stores the ciphertext, nonce, tag, HPKE encapsulated key, wrapped DEK, suite identifiers, and handler key ID. It never stores the plaintext DEK.

For cross-organization reassignment, the authorized current handler unwraps the DEK and HPKE-wraps only the same DEK to the destination organization's published key. The signed transfer binds old/new handler IDs, old/new key IDs, complaint ID, ciphertext hash, actor, reason, and time. There is no universal escrow key.

11. Complaint commitment and attachments

The client creates a manifest containing protocol version, matter ID/version, NFC-normalized text, and ordered attachment records. Each public-safe attachment record binds zero-based index, byte length, allowlisted MIME type, sanitized NFC filename, and SHA-256 of the final sanitized bytes.

```
complaintCommitment = SHA-256(salt32 || CBOR(manifest))
```

The salt is random 32 bytes and remains in the private package/user receipt material. The public log contains only the commitment. Encrypted originals may be included in the private package with distinct hashes and safety labels; they MUST NOT be substituted for the committed derivative without producing a new versioned event.

12. Receipt

The unsigned receipt contains protocol version, random 16-byte receipt ID, random 32-byte receipt nonce, complaint ID, matter ID/version, complaint commitment, acceptance time in Unix milliseconds, initial log-entry hash, and receipt signing-key ID. It excludes entitlement serial, person commitment, membership root, epoch, nullifier, challenge, IP, and user-agent data.

```
signature = Ed25519.Sign(receiptSK,  
  "CYBER-CIPHER/v1/receipt\x00" || CBOR(unsignedReceipt))
```

The signed receipt is itself deterministic CBOR. The client verifies strict canonical encoding, key ID, signature, matter and commitment equality, and clock plausibility before accepting it.

13. Transparency log and witnesses

Use RFC 6962-style domain separation:

```
leafHash = SHA-256(0x00 || CBOR(logEntry))  
nodeHash = SHA-256(0x01 || left32 || right32)
```

A signed tree head contains protocol version, tree size, root hash, timestamp, previous finalized tree-head hash, and log key ID. The log operator signs the prefixed deterministic encoding. Each witness independently downloads/validates new entries, verifies consistency from its last tree head, and signs the exact tree-head hash. A checkpoint is final with signatures from at least two of three configured witnesses.

The complaint receipt may initially bind the accepted log-entry hash before the containing checkpoint is final. The client MUST later retrieve an inclusion proof and a final witnessed tree head. Failure to finalize within the declared objective is visible and alertable; it does not invalidate the receipt signature.

14. Anonymous mailbox

For each complaint, generate followUpSeed32 and mailboxId16.

```
mailboxAuthSeed = HKDF(mailboxId16, followUpSeed32,  
  "CYBER-CIPHER/v1/mailbox-auth", 32)
```

```
mailboxX25519Seed = HKDF(mailboxId16, followUpSeed32,  
    "CYBER-CIPHER/v1/mailbox-hpke", 32)
```

The first output seeds Ed25519. The second is clamped as required for X25519. The CS stores only mailboxId, public keys, state, and ciphertext messages. Authentication challenges are random 16 bytes, valid for at most 60 seconds, and consumed only after successful signature verification.

Handler-to-user messages use the mailbox X25519 key with the same RFC 9180 HPKE suite. User-to-handler messages use the current handler organization key. Each encrypted message binds mailbox ID, monotonically increasing message number, direction, and previous-message hash as AAD. This supplies ordering and deletion detection; it is not a Double Ratchet and does not claim post-compromise security.

15. Prototype private-key files

Prototype RSA and service private keys stored outside an HSM MUST be encrypted PKCS#8 files in a dedicated secrets volume outside the source repository and database backups. File encryption uses AES-256-GCM with a random 12-byte nonce and a 256-bit KEK from the configured secret manager. AAD includes file-format version, key ID, matter ID/version where applicable, and algorithm identifier. File permissions MUST restrict access to the owning service identity.

16. Algorithm agility and future RSA accumulator

Every proof envelope declares scheme, circuitVersion, and artifactId. The application invokes a membership-provider interface with operations to publish state, obtain/update a witness, issue a root lease, produce a proof, and verify a proof.

The future RSA accumulator provider must define a trusted-modulus ceremony or class-group alternative, prime representative derivation, dynamic deletion/witness-update protocol, zero-knowledge membership statement, device-replacement binding to persistent P, proof-of-knowledge soundness, and compatibility test vectors. It MUST complete independent cryptographic review before replacing the Merkle provider.

Canonical CBOR Schemas and Test Vectors

1. Encoding profile

All security-relevant CBOR follows RFC 8949 deterministic encoding with these additional rules:

- definite lengths only;
- shortest integer and length encoding;
- map keys are unsigned integers and appear in deterministic encoded-key order;
- duplicate or unknown keys are rejected unless a schema explicitly marks an extension map;
- floats, tags, null, and text representations of binary data are forbidden;
- byte lengths are exact;
- UTF-8 text is NFC normalized before encoding;
- UUIDs are raw 16-byte RFC 4122 network-order values;
- timestamps are unsigned Unix milliseconds;
- field elements are canonical 32-byte big-endian values less than the BN254 scalar modulus;
- hashes and key IDs are raw 32-byte strings;
- signatures are raw fixed-width bytes.

The machine-readable CDDL is in `schemas/cyber-cipher-v1.cddl`. JSON in this project is diagnostic only and is never the signed wire representation.

2. Schema registry

Type	Purpose	Signed/encrypted form
entitlement-message	Blind RSA application message	Input to RFC 9474 preparation
entitlement-token	Spendable message/randomizer/signature tuple	Stored on client; sent once to CS
membership-checkpoint	Current active root	Ed25519 signed by IdA checkpoint key
proof-lease	Current-root challenge	Ed25519 signed by proof-session key
proof-envelope	Groth16 proof and ordered public inputs	Verified against allowlisted artifact
complaint-manifest	Text and sanitized attachment commitment	Salted SHA-256 commitment
private-complaint-package	Handler-readable content	AES-256-GCM encrypted
receipt	User-verifiable acceptance record	Ed25519 signed by CS receipt key
log-entry	Public tamper-evident event	RFC 6962 leaf hash
signed-tree-head	Log checkpoint	Ed25519 plus witness signatures
artifact-manifest	Reproducible circuit identity	SHA-256 content address

3. Integer key assignments

Keys are local to each map. They are never inferred from field names.

Entitlement message

Key	Field	Type
1	protocolVersion	uint, value 1
2	matterKeyId	bstr32
3	serial	bstr16
4	personCommitment	field32

Membership checkpoint unsigned body

Key	Field	Type
1	protocolVersion	uint
2	epoch	uint64
3	root	field32
4	previousCheckpointHash	bstr32; all-zero only at genesis
5	publishedAt	uint64 milliseconds
6	updateBatchHash	bstr32
7	signingKeyId	bstr32

The signed wrapper uses key 1 for the unsigned body and key 2 for the 64-byte Ed25519 signature.

Proof lease unsigned body

Key	Field	Type
1	protocolVersion	uint
2	challengeId	bstr16
3	purpose	uint: 1 complaint, 2 vote
4	epoch	uint64
5	membershipRoot	field32
6	issuedAt	uint64 milliseconds
7	expiresAt	uint64 milliseconds
8	serverNonce	bstr32
9	signingKeyId	bstr32

Complaint manifest

Key	Field	Type
1	protocolVersion	uint
2	matterId	bstr16
3	matterVersion	uint32
4	text	NFC UTF-8 text
5	attachments	ordered array of attachment records

Attachment record keys are 1=index, 2=size, 3=MIME, 4=sanitized filename, and 5=SHA-256(final sanitized bytes).

Receipt unsigned body

Key	Field	Type
1	protocolVersion	uint
2	receiptId	bstr16
3	receiptNonce	bstr32
4	complaintId	bstr16
5	matterId	bstr16
6	matterVersion	uint32
7	complaintCommitment	bstr32
8	acceptedAt	uint64 milliseconds
9	logEntryHash	bstr32
10	receiptKeyId	bstr32

The signed wrapper is {1: unsigned-body, 2: signature64}.

Log entry

Key	Field	Type
1	protocolVersion	uint
2	eventId	bstr16
3	eventType	uint registry value
4	complaintId	bstr16 or omitted for non-complaint events
5	matterId	bstr16 or omitted
6	matterVersion	uint32 or omitted
7	eventCommitment	bstr32
8	occurredAt	uint64 milliseconds
9	actorClass	uint registry value
10	reasonCode	uint or omitted

Public event commitments MUST be purpose-specific hashes of private event objects. They MUST NOT be raw hashes of predictable NICs, entitlement serials, or membership leaves.

4. Test-vector generation

`test-vectors/generate_vectors.py` is the normative generator for the included examples. It contains a small deterministic-CBOR encoder rather than relying on a library's default mode. Run:

```
python test-vectors/generate_vectors.py
```

It writes `test-vectors/cyber-cipher-v1-vectors.json` containing:

- encoded entitlement message and SHA-256 hash;
- encoded complaint manifest, fixed salt, and complaint commitment;
- encoded log entry and RFC 6962 leaf hash;
- unsigned receipt bytes, signing input, deterministic Ed25519 public key/signature, and complete receipt bytes;
- unsigned membership checkpoint bytes, signing input, public key/signature, and checkpoint hash;
- hash-to-field framing examples and all Poseidon domain constants;
- negative cases that strict decoders/verifiers must reject.

Fixed private seeds exist only to make the vectors reproducible and MUST NEVER be used outside tests.

5. Required conformance assertions

A conforming implementation MUST reproduce every hexadecimal byte string and decimal field value in the vector file. It MUST also reject:

1. non-minimal integer encodings;
2. indefinite-length arrays/maps/byte strings;
3. duplicate or unknown map keys;
4. a field element equal to or greater than q ;
5. an all-zero scalar where a secret must be non-zero;
6. a receipt whose signature is valid over a differently ordered/noncanonical encoding;
7. a proof envelope whose public-input order differs from the artifact manifest;
8. an entitlement with a 15- or 17-byte serial;
9. a lease longer than 60 seconds or accepted after expiration;
10. a receipt containing a serial, nullifier, root, epoch, or person commitment;
11. Unicode text that was not normalized to NFC before encoding;
12. attachment arrays with duplicate or non-contiguous indices.

6. Compatibility policy

Adding a required field, changing a key number, changing a cryptographic derivation, or changing the meaning/order of proof signals requires a new protocol or circuit version. Optional extensions must live inside an explicitly defined extension map in a future schema; version 1 has no generic extension maps. Version-1 decoders fail closed on unknown keys.

Database and API Contracts

1. Contract principles

- Identity, complaint, transparency-log, witness, and handler data live in separate databases with separate service credentials and backups.
- There are no cross-zone foreign keys, shared ORM models, database links, analytics joins, or shared administrator credentials.
- Binary protocol objects are stored as validated canonical CBOR plus indexed, non-sensitive columns where required.
- Client-supplied identifiers are random UUIDv4/128-bit values. Internal sequential IDs are never exposed across trust zones.
- All write APIs require idempotency keys. Repeating the same key with different request bytes is rejected.
- Public binary APIs use `application/cbor`; diagnostic JSON endpoints, where enabled, are not signature inputs.
- Date/time database columns are UTC. Cryptographic wire timestamps are Unix milliseconds.
- The reference DDL is `database/reference-schema.sql`; physical deployments split its sections into different database clusters.

2. Identity database

`enrollment`

Stores one identified eligibility record. `nic_lookup` is a keyed HMAC of normalized NIC for uniqueness; `nic_ciphertext` is encrypted for operational/legal use. The HMAC key and encryption key are independent KMS keys.

Required fields: internal enrollment ID, NIC lookup/ciphertext, eligibility status, `person_anchor`, active device hash/index, recovery ID/public key/generation, enrollment timestamps, and row version.

Forbidden fields: complaint ID/content/commitment, entitlement serial/signature, receipt/mailbox/vote/nullifier.

`matter_issuance`

Stores only (`enrollment_id`, `matter_id`, `matter_version`, `completed_at`), with a unique constraint over the first three values. It MUST NOT store the blinded request, entitlement message, serial, person commitment, blind signature, or final signature.

Membership tables

`membership_leaf` stores index, current field value, state, and update epoch. `membership_checkpoint` stores signed canonical checkpoint bytes and hashes. `membership_update` stores the append/revoke operations needed to reconstruct each root. Index allocation uses a transactionally locked monotonic sequence and never searches for empty slots.

Recovery tables

`recovery_challenge` stores challenge ID, recovery record reference, signed challenge bytes, expiry, consumed time, and bounded attempt count. Consumed/expired rows are purged within 24 hours. A successful recovery locks the enrollment and recovery rows and commits credential rotation with membership updates.

3. Complaint database

`matter`

Stores public matter ID/version, title, window, RSA SPKI/fingerprint, circuit artifact IDs, handler organization/key ID, publication time, and lifecycle status. (`matter_id`, `version`) and `matter_key_id` are unique.

`proof_session`

Stores random challenge ID, purpose, root/epoch, signed lease, issue/expiry times, and consumption state. It stores no client network metadata. Expired sessions are purged within 24 hours.

spent_entitlement and used_nullifier

Uniqueness tables enforce one spend. The entitlement table keys on (matter_key_id, serial). The nullifier table keys on (scope_type, scope_id, nullifier). Insertion occurs in the same serializable transaction as complaint/vote acceptance. Raw proofs and proof witnesses are not retained; an optional verification audit record stores artifact ID, verification result, and hash of the canonical proof envelope.

complaint

Stores random complaint ID, matter reference, public commitment, ciphertext object reference/hash/size, AES-GCM nonce/tag, HPKE encapsulated key/wrapped DEK/suite, handler organization/key ID, status, mailbox public fields, accepted time, and initial log-entry hash.

Forbidden fields: NIC, phone, email, person/device hashes, membership index/path, person secret, device secret, recovery ID, source IP, TLS fingerprint, advertising ID, entitlement person commitment, serial/nullifier in the receipt view.

Lifecycle and access

case_event is append-only and stores a signed private event plus its public commitment/log reference.

case_access_event records authorized or denied accesses without copying complaint plaintext. Status is derived from the latest valid transition and cached in complaint; direct status updates outside the transition procedure are denied.

Mailbox

mailbox stores random mailbox ID and the two public keys. mailbox_message stores monotonically numbered ciphertexts, direction, previous-message hash, handler key ID as relevant, and creation time. Private mailbox seeds are forbidden.

4. Public log database

log_leaf is immutable and append-only by database role. It stores leaf index, exact canonical log-entry bytes, leaf hash, and insertion time. tree_head stores operator-signed heads, and witness_signature stores one signature per configured witness/head. No update or delete privilege is granted to the application role.

Log replication and witness verification do not depend solely on database replication: each witness independently reconstructs the Merkle tree from public leaves.

5. API surface

The machine-oriented outline is api/openapi.yaml. Required endpoint groups follow.

Identity and membership

Method/path	Authentication	Purpose
POST /ida/v1/enrollments	Identified enrollment session	Validate NIC/contact and create person/device membership
POST /ida/v1/matters/{id}/{version}/blind-issuance	Enrolled device session	Submit blinded RSA request; one completed issuance per NIC/matter version
GET /public/v1/membership/checkpoints/current	None	Current signed membership checkpoint
GET /public/v1/membership/deltas?afterEpoch=	None	Public updates for local tree/path reconstruction
POST /ida/v1/recovery/challenges	Recovery ID plus abuse controls	Issue signed five-minute challenge
POST /ida/v1/recovery/complete	Recovery signature	Atomic device/recovery rotation

Complaints

Method/path	Authentication	Purpose
GET /public/v1/matters	None	Published matter metadata and keys
POST /complaints/v1/proof-sessions	None	Current-root 60-second signed lease
POST /complaints/v1/complaints	RSA token plus Groth16 proof	Atomic anonymous submission
GET /public/v1/receipts/{receiptId}/inclusion	None; privacy-preserving bulk alternative required	Inclusion proof and finalized tree head

The per-receipt endpoint is convenient but can reveal which receipt a client checks. Clients SHOULD use bulk checkpoint/leaf downloads through Tor or privacy relay. The UI must disclose this distinction.

Mailbox

Method/path	Authentication	Purpose
POST /mailboxes/v1/{mailboxId}/challenges	None plus abuse controls	Issue short-lived mailbox challenge
GET /mailboxes/v1/{mailboxId}/messages	Mailbox challenge signature	Retrieve ciphertext messages
POST /mailboxes/v1/{mailboxId}/messages	Mailbox signature	Send encrypted reply

Handler and reviewer

Method/path	Authentication	Purpose
GET /handler/v1/cases	WebAuthn-authenticated staff	Assigned case list
GET /handler/v1/cases/{id}	Assigned-handler authorization	Ciphertext/envelope and case metadata
POST /handler/v1/cases/{id}/events	Handler plus action signature	Lifecycle event
POST /handler/v1/cases/{id}/transfer	Authorized current handler	Cross-organization DEK rewrap
POST /handler/v1/cases/{id}/redactions	Handler	Proposed public derivative
POST /review/v1/redactions/{id}/decision	Independent reviewer	Approve/reject redaction

Community and audit

Method/path	Authentication	Purpose
GET /public/v1/complaints/{id}	None	Approved redacted public record
POST /community/v1/proof-sessions	None	Current-root vote lease
POST /community/v1/complaints/{id}/votes	Vote proof	Atomic anonymous vote
GET /public/v1/complaints/{id}/vote-result	None	Aggregate outcome after policy rules
POST /auditor/v1/complaints/{id}/assessment	Auditor WebAuthn	Signed comparison/freeze/escalation
GET /log/v1/entries	None	Paginated/full log download
GET /log/v1/tree-heads/latest	None	Latest operator and witness signatures
GET /log/v1/proofs/inclusion	None	RFC 6962 inclusion proof
GET /log/v1/proofs/consistency	None	RFC 6962 consistency proof

6. Atomic submission contract

The following operations are one serializable transaction or one transaction plus a durable outbox whose receipt binds the committed outbox event:

1. lock challenge and verify not consumed/expired;
2. verify matter/window, entitlement, proof, ciphertext metadata, and mailbox keys;
3. insert unique spent serial;
4. insert unique complaint nullifier;
5. insert complaint and initial private event;
6. reserve/append public log entry through the durable log outbox;
7. insert receipt record and mark challenge consumed;
8. commit;
9. sign and return the canonical receipt using the committed values.

If steps 3-8 fail, the serial, nullifier, and challenge remain unconsumed. Idempotent replay of a committed request returns the same receipt; a different request under the same idempotency key is rejected.

7. Uniform errors

Unauthenticated cryptographic failures return a common status and body, for example HTTP 400 with code SUBMISSION_NOT_ACCEPTED. Logs may record a privacy-safe internal category but MUST NOT record secrets or raw tokens. Rate-limit responses MUST not reveal whether a serial/nullifier exists.

Authenticated administrative APIs may return precise authorization and validation errors because the actor is already identified and audited.

8. State transitions

Allowed case transitions are policy-versioned. The default graph is:

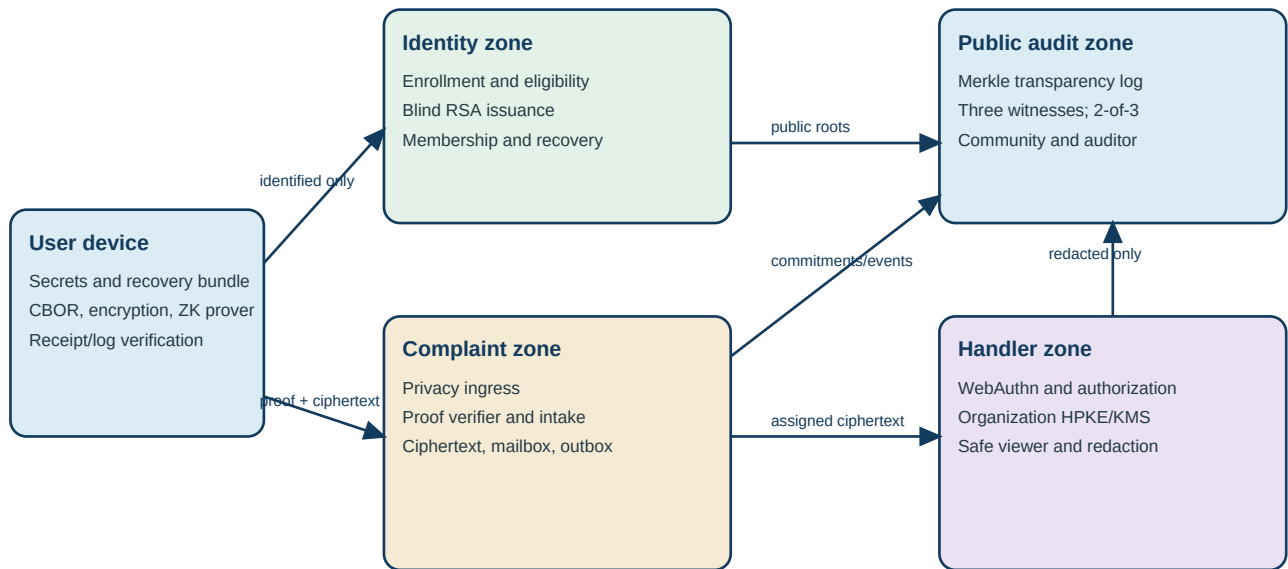
```
SUBMITTED -> ACKNOWLEDGED -> ASSIGNED -> UNDER_REVIEW
UNDER_REVIEW -> ACTION_REQUIRED -> UNDER_REVIEW
UNDER_REVIEW -> RESOLVED | REJECTED | CLOSED
RESOLVED | REJECTED | CLOSED -> APPEALED -> UNDER_REVIEW
```

SUPPRESSED and PUBLIC_FROZEN are public-visibility flags, not destructive case states. Every transition requires expected previous state, actor, reason code, timestamp, and signature. Optimistic concurrency prevents two conflicting transitions.

9. Retention and erasure jobs

Retention jobs operate inside each zone and append privacy-safe deletion events before removing ciphertext/object data. They never delete log leaves or finalized tree heads. Evidence-object deletion is verified by listing and attempting retrieval; encryption-key destruction is recorded separately. Legal holds override timed deletion and require a signed, attributable hold event.

Service Architecture



1. Architectural style

The prototype uses separately deployable containers organized by trust zone, not a single process and not a large production microservice estate. Separation is driven by privacy boundaries: identity services must not share storage, credentials, or telemetry with complaint services. Docker Compose is the initial runtime; production can map the same boundaries to Kubernetes or isolated virtual machines after operational review.

The reference diagram is also available as `diagrams/service-architecture.mmd`.

Diagram source is preserved in the editable Markdown package.

2. Component responsibilities

Mobile app

- Generates and protects P, D, recovery and mailbox secrets.
- Performs CBOR encoding, commitment calculation, RSA blind protocol, evidence preprocessing, Groth16 proving, encryption, receipt verification, and log-proof verification.
- Maintains a local copy of public membership deltas sufficient to construct a Merkle path without asking the IdA for a complaint-specific lookup.
- Supports direct TLS and Tor/Orbot-style routing; clearly identifies which mode is active.
- Exports an encrypted recovery bundle only after warning the user about loss/coercion implications.

Enrollment and issuance API

- Owns NIC eligibility integration and identified device sessions.
- Stores person anchor/device hash, issuance facts, and recovery public state.
- Performs blind RSA signing only after the unique issuance row is secured.
- Has no network route to complaint storage and no complaint-oriented endpoint.

Membership tree builder

- Serializes leaf allocation/revocation.

- Applies pending changes every 30 seconds if any exist.
- Produces a signed checkpoint and public delta package.
- Sends the checkpoint hash to the transparency outbox without identity data.

Privacy ingress

- Terminates public/Tor traffic with strict request-size and time limits.
- Removes or isolates network identifiers before forwarding.
- Uses coarse, short-lived abuse counters that cannot be joined to complaint IDs.
- Does not perform content analytics or inject third-party scripts.

Proof verifier

- Holds an allowlist mapping artifact IDs to immutable verification keys and public-input layouts.
- Verifies leases, RSA entitlements, Groth16 proofs, and canonicity.
- Is stateless with respect to identity and cannot query IdA enrollment records.

Atomic intake

- Performs the serializable consume/store/log-outbox/receipt transaction.
- Encrypts no user plaintext because ciphertext arrives from the client.
- Signs receipts only from committed database values.
- Returns uniform public errors.

Handler portal

- Requires WebAuthn and authorization to the assigned organization/case.
- Retrieves ciphertext and invokes organization-controlled KMS/HSM unwrap.
- Opens originals only in a sandboxed safe viewer; normal UI uses sanitized derivatives.
- Signs all accesses, transitions, exports, redactions, and transfers.

Transparency log and witnesses

- Log accepts allowlisted canonical public events from durable outboxes.
- It builds RFC 6962-style trees and signs tree heads.
- Each witness independently downloads leaves, verifies the previous consistency proof, rebuilds the root, and signs the exact head.
- The public verifier accepts finality only with two configured witness signatures.

Community and auditor services

- Publish only independently approved redacted content.
- Verify the dedicated vote circuit, enforce unique vote nullifiers, and aggregate results.
- Compare aggregate assessment with handler outcome; never infer identity and never automatically close a complaint.

3. Primary sequences

Complaint submission

Diagram source is preserved in the editable Markdown package.

Device recovery

Diagram source is preserved in the editable Markdown package.

4. Deployment topology

Prototype

- One Docker Compose project with separate networks: public, identity, complaint, handler, and audit.
- Distinct PostgreSQL instances/volumes for identity, complaint, and log zones.
- S3-compatible object storage for encrypted evidence; object credentials limited to complaint/handler roles.
- A message broker or transactional outbox poller for log publication. The database outbox is the source of durability, not an in-memory queue.
- Three witness containers configured with separate signing keys and storage; demonstration documentation must state that true independence requires different operators/hosts.
- Secret-manager emulator for development; encrypted PKCS#8 files mounted read-only into only the owning container.
- No production secrets, real NIC records, or real complaints in the prototype environment.

Production target

- Separate cloud accounts/projects or independently administered hosts for identity, complaint, and audit zones.
- Managed PostgreSQL with encrypted point-in-time recovery and independent backup credentials.
- HSM/KMS keys with service identity, purpose restrictions, two-person destructive controls, and audit export.
- Private handler connectivity and organization-owned X25519 key custody.
- Tor onion service or reviewed privacy relay terminating into the privacy ingress.
- Witnesses operated by independent institutions and exposed over independently controlled domains/networks.

5. Recommended implementation stack

This is a pragmatic prototype baseline, not a cryptographic requirement:

- Monorepo with TypeScript for public/backend services and web portals.
- Flutter mobile client with native bindings for platform keystores and Rapidsnark-compatible proving.
- Circom 2 circuits plus pinned build container.
- PostgreSQL for transactional stores; S3-compatible encrypted object storage.
- A small Rust or similarly memory-safe shared crypto service/library for strict CBOR, HPKE, signature, and hash operations where mature reviewed libraries are available.
- React-based handler, reviewer, community, and auditor portals.
- OpenTelemetry-compatible metrics with content and high-cardinality user identifiers prohibited.

Every cryptographic dependency and compiler is pinned by digest and appears in the software bill of materials and artifact manifest.

6. Network and data controls

- Default deny between zones. Complaint services cannot call identified enrollment APIs except public matter/checkpoint distribution.
- Public checkpoint/matter artifacts are replicated through a static/CDN origin with privacy-reviewed logs.
- Egress from proof/intake services is restricted to object storage, complaint DB, KMS, and log outbox destinations.
- Backups inherit zone separation; a backup operator for one zone cannot restore another.
- Debug endpoints, heap dumps, request-body tracing, and database statement logging are disabled in security-sensitive services.
- Clock synchronization alerts at configured skew; expired leases are rejected using server monotonic time plus signed wall time.

7. Availability and failure behavior

Failure	Required behavior
IdA unavailable	Existing users can submit using published keys/current cached state until lease service/root policy prevents it; new enrollment/issuance/recovery pauses.
Complaint DB unavailable	No receipt is issued and token/challenge are not consumed. Client retries idempotently.
Object store unavailable	Transaction does not accept complaint unless ciphertext durability is confirmed.
Log unavailable	Durable outbox preserves accepted event; receipt is issued with pending inclusion and UI warns until finalization.
One witness unavailable	Remaining two may finalize.
Two witnesses unavailable/disagree	Tree head remains non-final; acceptance may continue only under declared degraded-mode policy and produces a prominent audit alert.
Handler KMS unavailable	Complaint remains accepted/encrypted; handler access is delayed, not bypassed.
Current root changes during proof	Existing signed lease remains valid only through its 60-second expiry; server verifies the leased root was current when issued and not administratively invalidated for key compromise.

8. Observability

Allowed metrics include aggregate request count, latency buckets, CPU/memory, queue depth, object-store errors, proof-verification outcomes by broad category, checkpoint age, witness quorum, and membership capacity. Labels **MUST NOT** include complaint ID, receipt ID, mailbox ID, serial, nullifier, recovery ID, NIC, IP, user agent, or ciphertext hash.

Security audit records are signed/append-only within their zone. Public events expose only the commitments and metadata allowed by the log schema.

9. Architecture acceptance conditions

- No runtime credential permits a database join across identity and complaint zones.
- The complaint service can be given full IdA and CS database dumps and still has no deterministic complaint-to-NIC key.
- The CS cannot decrypt a complaint without the handler organization's private key.
- Revoking one device changes the current membership root and makes an old proof fail after lease expiration.
- Deleting a private complaint row does not rewrite the log; receipt/log inclusion still proves prior acceptance while retrieval failure exposes the operational loss.
- Log consistency proofs detect a rewritten public history, not deletion from an unrelated private database.

Phased Implementation Roadmap

1. Delivery strategy

Build vertical security slices, not UI-first mock functionality. Each phase ends with executable tests and an explicit exit gate. Production pilots are prohibited until the external-review gate.

The critical path is:

Diagram source is preserved in the editable Markdown package.

2. Phase 0 - Repository and governance baseline

Deliverables:

- Accept this specification under version control.
- Create a monorepo layout for mobile, web, services, circuits, shared schemas, infrastructure, and test vectors.
- Add decision records for privacy boundary, proof statement, RSA suite, recovery, retention, and log finality.
- Define code owners so circuit/crypto/schema changes require security review.
- Configure secret scanning, signed commits/releases, dependency lockfiles, SBOM generation, and CI without production secrets.
- Build a synthetic-data policy; real NICs and complaints are forbidden in developer environments.

Exit gate: schemas and vector generator run in CI; no unresolved TBD changes a security statement or wire format.

3. Phase 1 - Canonical encoding and cryptographic core

Build first:

- Strict deterministic-CBOR encoder/decoder in the shared supported languages.
- Domain-separation and HashToField functions.
- Ed25519 signing/verification and DER-SPKI key identifiers.
- AES-256-GCM and HPKE envelope helpers with fixed suite identifiers.
- Receipt, checkpoint, lease, log-leaf, and artifact-manifest libraries.
- Cross-language conformance against `cyber-cipher-v1-vectors.json`.

Do not implement custom RSA padding, Poseidon, Ed25519, AES, HPKE, or Groth16 primitives. Wrap reviewed libraries and add protocol-level validation.

Exit gate: every language reproduces all positive vectors and rejects all negative vectors; fuzzers find no parser crash or ambiguous acceptance.

4. Phase 2 - Identity, membership, and recovery vertical slice

- Implement synthetic NIC registry adapter and identified enrollment flow.
- Generate/store `personAnchor`, device hash, recovery public state, and encrypted user backup.
- Implement depth-16 Poseidon tree with monotonic allocation and fixed empty leaves.
- Publish 30-second signed checkpoints and deltas.
- Implement five-minute recovery challenge and atomic old-leaf revocation/new-index allocation/recovery rotation.
- Build a client path updater and checkpoint-chain verifier.

Demo: enroll, produce current path, lose device, recover using seed, observe old leaf revoked and new leaf active after the next checkpoint.

Exit gate: recovery concurrency tests prove that at most one replacement succeeds and no index is reused.

5. Phase 3 - Matter registry and RSA blind entitlement

- Implement matter UUID/version lifecycle and 24-hour prepublication rule.
- Generate FIPS-compatible RSA-3072 keys with exponent 65537.
- Encrypt prototype PKCS#8 files using secret-manager KEK; prevent repository/database inclusion.
- Implement RFC 9474 randomized blind issuance and local final verification.
- Enforce one issuance fact per enrollment/matter version.
- Implement close/retire job and evidence of private-key destruction.

Demo: issue a blind entitlement, show that the IdA database cannot recognize the unblinded token, and prove a second issuance for the same matter is rejected.

Exit gate: official RFC/library test vectors pass; cross-protocol/key-reuse tests fail closed.

6. Phase 4 - Circuits, setup, and mobile proving

- Implement complaint and vote circuits with exact public-input order.
- Unit-test every constraint and assert constraint counts.
- Pin Circom/circomlib/build container.
- Select and verify the Powers-of-Tau transcript.
- Run three independent phase-two contributions per circuit and publish transcripts/hashes.
- Produce R1CS, WASM/native prover assets, proving keys, verification keys, and deterministic artifact manifests.
- Integrate a Rapidsnark-compatible prover on representative Android/iOS devices.

Exit gate: invalid path, wrong P, wrong D, wrong r, changed matter/serial/commitment/challenge, and revoked leaf all fail. Proving performance and memory fit the supported device baseline.

7. Phase 5 - Encrypted atomic complaint submission

- Implement evidence allowlist, size limits, client sanitization, NFC normalization, manifest, and salted commitment.
- Implement per-complaint AES-GCM and handler HPKE wrapping.
- Implement 60-second proof-session lease service.
- Build the verifier allowlist and fixed verification order.
- Implement the serializable submission transaction and idempotency behavior.
- Implement randomized deterministic-CBOR receipt generation and mobile verification.
- Add direct TLS and test-only Tor routing; disable body/network-identifier logs.

Exit gate: a successfully returned receipt always corresponds to one durable ciphertext and one durable log outbox entry; every failed transaction leaves serial/nullifier/challenge reusable.

8. Phase 6 - Transparency log, witnesses, and public verifier

- Implement RFC 6962-style leaf/node hashes, tree construction, inclusion and consistency proofs.
- Sign tree heads and deploy three witness implementations/configurations.
- Require 2-of-3 finality and implement tree-head gossip.
- Build a downloadable public log and open-source offline verifier.
- Implement mobile background inclusion verification and plain-language status.

Demo: show valid receipt/inclusion, mutate a leaf, present divergent heads, and withhold a witness.

Exit gate: mutation/fork proofs are rejected, one unavailable witness does not stop finality, and two unavailable/disagreeing witnesses produce a visible non-final state.

9. Phase 7 - Handler workflow and anonymous mailbox

- Add WebAuthn staff enrollment and least-privilege authorization.
- Build case assignment, signed access events, state transitions, SLA timers, extensions, appeal, and suppression without deletion.
- Integrate handler KMS/HPKE unwrap and sandboxed evidence viewing.
- Implement signed DEK rewrap for cross-organization transfer.
- Implement complaint-specific mailbox keys, signed challenges, HPKE messages, ordering hashes, and encrypted user recovery bundle.

Exit gate: an unassigned handler cannot retrieve/decrypt a case; every allowed/denied access is attributable; transfer exposes only the DEK to authorized handler environments.

10. Phase 8 - Redaction, community voting, and auditor comparison

- Implement opt-in publication, automated PII scanning, handler draft, and independent reviewer approval.
- Publish redacted derivative provenance and commitment.
- Integrate vote circuit, current-root lease, unique vote nullifier, seven-day window, quorum 10, and 60% rule.
- Implement comparison statuses and auditor freeze/escalation workflow.
- Ensure community results never automatically close/delete a private complaint.

Exit gate: no raw attachment/plaintext enters the public store; a user cannot vote twice for one complaint, including after device replacement preserving P.

11. Phase 9 - Hardening and controlled pilot

- Complete the full test plan, privacy data inventory, DPIA-style review, and accessibility/language testing.
- Run backup/restore, key rotation, key compromise, witness outage, database failover, and recovery exercises.
- Conduct SAST, dependency, container, IaC, DAST, fuzz, load, penetration, and external cryptographic reviews.
- Resolve all critical/high findings; document accepted medium/low risks.
- Name independent witness operators and handler key custodians.
- Validate retention, appeals, moderation, and incident policies with legal/governance owners.

Exit gate: formal production-readiness sign-off. Until then, use synthetic participants and content only.

12. Suggested repository layout

```
apps/  
  mobile/  
    handler-web/  
    reviewer-web/  
    community-web/  
    auditor-web/  
services/  
  ida/  
    membership/  
    recovery/  
    complaint-intake/  
    proof-verifier/  
    mailbox/  
    transparency-log/  
    witness/  
circuits/  
  complaint-membership/  
  community-vote/  
packages/  
  protocol-cbor/  
  crypto-profile/  
  test-vectors/  
infra/  
  compose/  
  production/  
docs/
```

13. First three implementation iterations

Iteration 1

Repository, CDDL, vector CI, key-ID/signature helpers, log hash helpers, and database migrations.

Iteration 2

Synthetic enrollment, membership tree/checkpoint/delta publication, client path reconstruction, and recovery transaction.

Iteration 3

Matter publication, RSA blind issuance, initial complaint circuit, and end-to-end proof verification with synthetic ciphertext.

Do not start public UI polishing before these iterations pass their security tests; wire-format or proof-statement changes after UI integration create expensive rework.

14. Completion definition

A feature is complete only when its protocol/schema is versioned, positive and negative tests exist, privacy-forbidden fields are tested, logs contain no sensitive values, failure is atomic, documentation matches behavior, and the acceptance matrix traces it to an SRS requirement.

Security and Acceptance-Test Plan

1. Purpose and environments

This plan verifies the prototype's functional, cryptographic, privacy, and operational claims. Tests use synthetic identities and content. Production secrets and real complainant data are prohibited.

Environments:

- **Unit/conformance:** isolated libraries and circuits.
- **Integration:** all containers with separate databases and development keys.
- **Adversarial:** instrumented environment where testers receive selected databases, credentials, traffic captures, or device images.
- **Recovery/operations:** backup copies and disposable infrastructure.
- **Device lab:** representative low/mid/high Android and iOS devices.

Evidence retained for every test: build/artifact IDs, configuration hash, start/end time, tester, sanitized logs, result, linked defect, and approval.

2. Release gates

Gate	Requirement
G1 Protocol conformance	All CBOR/hash/signature vectors match across languages; negative vectors rejected.
G2 Cryptographic correctness	Circuit, RSA blind issuance, recovery, HPKE, receipt, and log tests pass; no unresolved critical/high cryptographic finding.
G3 Privacy separation	Forbidden-field scans and database-join exercise pass; telemetry contains no prohibited identifier/content.
G4 Transaction integrity	Fault injection shows no consumed credential without accepted durable record/receipt and no receipt without durable record/outbox.
G5 Verifiability	Receipt, inclusion, consistency, witness quorum, and fork tests pass.
G6 Authorization	Handler/reviewer/auditor access tests pass with WebAuthn and attributable events.
G7 Resilience	Backup restore, node loss, queue replay, key rotation, and outage drills pass.
G8 Product quality	Accessibility, trilingual UI, usability, performance, and retention tests pass.
G9 External approval	Independent cryptographic review, penetration test, privacy review, and legal/governance approval completed.

G1-G8 are required for a synthetic-data demonstration. G9 is additionally required before any real pilot.

3. Encoding and key tests

ID	Test	Expected result
ENC-01	Run vector generator twice in clean environments.	Byte-identical JSON and artifact IDs.
ENC-02	Encode vectors in every supported language.	Exact CBOR hex and hashes match.
ENC-03	Supply non-minimal integers, indefinite lengths, duplicate/unknown keys, floats, and tags.	Strict rejection before signature/proof processing.
ENC-04	Supply field values q, q+1, wrong width, and forbidden zero secrets.	Rejected as noncanonical/invalid.
ENC-05	Supply decomposed Unicode and reordered attachment indices.	Rejected or normalized before commitment; never ambiguously accepted.
KEY-01	Compute key IDs from DER SPKI in all languages.	Full 32-byte fingerprints match.
KEY-02	Use one service's Ed25519 key to sign another object type.	Verification fails because key allowlist/prefix differs.
KEY-03	Attempt RSA matter-key use for ordinary signature/TLS/another matter.	Policy/API rejects use.
KEY-04	Scan repository, images, DB dumps, and logs for private keys/test-to-production key reuse.	No production private material; test seeds clearly isolated.

4. Enrollment, membership, and recovery

ID	Test	Expected result
MEM-01	Enroll a valid synthetic NIC.	One enrollment, one unique active leaf/index, signed next checkpoint.
MEM-02	Enroll same NIC/device concurrently.	Exactly one active enrollment; race safely rejected/idempotent.
MEM-03	Fill/revoke leaves and enroll again.	Revoked index is never reused; allocation is monotonic.
MEM-04	Recompute every root from public deltas.	Root equals signed checkpoint for every epoch.
MEM-05	Tamper with checkpoint root, epoch, predecessor, or signature.	Client rejects and raises fork/integrity warning.
MEM-06	Generate proof for active leaf/current root.	Verification succeeds.
MEM-07	Revoke leaf, wait for root update/lease expiry, reuse old proof/path.	Verification fails.
REC-01	Recover with correct seed and five-minute challenge.	Old leaf revoked; fresh index/device/recovery credential activated atomically.
REC-02	Replay consumed recovery challenge.	Rejected; no state change.
REC-03	Use expired, altered, or wrong-generation challenge.	Rejected uniformly; no state change.
REC-04	Submit two concurrent valid recovery completions.	Exactly one succeeds.
REC-05	Inject failure after revocation but before activation.	Transaction rolls back; original active state remains.
REC-06	Vote before and after legitimate recovery.	Same persistent P yields same complaint vote nullifier; second vote rejected.
REC-07	Restore user mailbox bundle and rotate recovery seed.	Existing mailbox seeds recover and bundle re-encrypts; old seed cannot decrypt new bundle.

5. RSA entitlement tests

ID	Test	Expected result
RSA-01	Generate matter key.	3072-bit modulus, exponent 65537, fixed PSS/SHA-384 parameters, unique fingerprint.
RSA-02	Complete blind issuance and final verification.	Valid token; IdA never receives final message/signature.
RSA-03	Compare stored issuance data with later tokens across a large synthetic sample.	No deterministic token identifier stored by IdA.
RSA-04	Request second issuance for same NIC/matter version.	Rejected by unique issuance fact.
RSA-05	Change matter key ID, serial, commitment, message randomizer, signature, PSS salt assumptions, or public key.	Verification fails.
RSA-06	Reuse blinding randomness in instrumented client.	Test harness detects forbidden reuse; production CSPRNG path never accepts caller-supplied randomness.
RSA-07	Submit token before open/after close.	Rejected with uniform public error.
RSA-08	Close matter and run retirement.	Private key destroyed within 24 hours; public key/metadata remain verifiable.

6. Circuit and proof tests

ID	Test	Expected result
ZK-01	Valid complaint witness and bound public inputs.	Proof verifies with allowlisted artifact.
ZK-02	Change P, D, r, one sibling/path bit, or root.	Proof generation/verification fails.
ZK-03	Use entitlement person commitment from a different P.	Proof fails.
ZK-04	Change matter, serial, complaint commitment, or challenge after proving.	Request-to-public-input comparison fails.
ZK-05	Use unknown artifact ID or verification key fetched from request-controlled URL.	Rejected; no dynamic untrusted key fetch.
ZK-06	Reorder public signals.	Rejected against manifest-defined ordering.
ZK-07	Submit malformed curve/proof points and noncanonical field encodings.	Rejected without crash or excessive resource use.
ZK-08	Valid vote proof for each of four choices.	Accepted once per person/complaint.
ZK-09	Reuse vote nullifier with another choice/proof.	Rejected atomically.
ZK-10	Audit R1CS constraints and setup transcripts.	Source/constraint/artifact hashes match manifest; all three contributions verify.

7. Submission, encryption, and receipt tests

ID	Test	Expected result
SUB-01	Valid token/proof/ciphertext submission.	One complaint, spent serial, nullifier, outbox event, and signed receipt.
SUB-02	Replay identical committed request with same idempotency key.	Same receipt returned; no duplicate records.
SUB-03	Reuse idempotency key with different bytes.	Conflict; no additional state.
SUB-04	Replay token under new challenge/request.	Uniform rejection; no second complaint.
SUB-05	Fail at every transaction boundary using fault injection.	Either complete commit or zero credential consumption/state change.
SUB-06	Inspect receipt fields.	Random receipt ID/nonce present; serial/nullifier/root/epoch/person commitment absent.
SUB-07	Modify any receipt field/signature or encode noncanonically.	Client rejects.
SUB-08	Decrypt handler package with assigned organization key.	Plaintext and AAD verify.
SUB-09	Attempt decryption with CS, wrong handler, wrong key version, changed AAD/ciphertext.	Fails without plaintext release.
SUB-10	Rewrap DEK to another organization through authorized transfer.	Ciphertext unchanged; new organization decrypts; signed transfer logged.
SUB-11	Attempt transfer by unauthorized actor or introduce universal escrow key.	Denied/test fails architecture policy.
SUB-12	Upload disallowed type, active PDF/script/archive, oversized file, decompression bomb, or MIME mismatch.	Rejected/quarantined within resource limits.
SUB-13	Upload JPEG with GPS EXIF.	Public derivative and committed bytes have no GPS metadata; original remains encrypted/restricted if retained.

8. Transparency and verification tests

ID	Test	Expected result
LOG-01	Append leaves and verify inclusion at multiple tree sizes.	RFC-style proofs validate; wrong leaf/index/path fails.
LOG-02	Verify consistency for increasing tree sizes.	Valid append-only histories pass; rewritten prefix fails.
LOG-03	Present two divergent signed heads.	Gossip/auditor reports split view; inconsistent head not silently accepted.
LOG-04	One witness unavailable.	Other two can finalize.
LOG-05	Only one witness signs or witnesses disagree.	Head remains non-final and alert is visible.
LOG-06	Modify/reorder a public log entry.	Leaf/root mismatch; signatures/proofs fail.
LOG-07	Delete private complaint row after final inclusion.	Receipt/log still prove prior acceptance; retrieval failure is reported. Do not incorrectly claim log consistency itself detects private-row deletion.
LOG-08	Block submission before receipt.	No proof of acceptance exists; documentation/UI make this limitation explicit.
LOG-09	Accept complaint while log temporarily down.	Durable outbox survives restart; later inclusion succeeds; pending state visible.
LOG-10	Download and verify full log without account.	Independent verifier reconstructs every finalized root.

9. Privacy and unlinkability tests

ID	Test	Expected result
PRIV-01	Give testers full IdA and CS database dumps and application logs.	No deterministic complaint-to-NIC mapping from stored application data.
PRIV-02	Automated schema/log scan for forbidden fields and representative secret patterns.	Zero findings in prohibited zones.
PRIV-03	Inspect ingress, CDN, APM, crash, analytics, and database statement logs.	No complaint-joinable IP, TLS fingerprint, user agent, body, token, nullifier, or mailbox secret.
PRIV-04	Submit two matters and complaint/vote from same person.	Public values are unlinkable except intentionally public case relationships.
PRIV-05	Compare blind issuance and spend timings under realistic traffic.	Report measured correlation risk; no claim of perfect resistance.
PRIV-06	Submit via Tor and direct TLS.	Both work; UI distinguishes privacy level; application stores neither source address with complaint.
PRIV-07	Query inclusion through direct endpoint and bulk/Tor mode.	Documentation and UI identify that direct lookup reveals lookup interest; bulk mode avoids complaint-specific query.
PRIV-08	Export receipt/recovery bundle.	Explicit coercion/loss warning and encryption required.

10. Mailbox, handler, community, and lifecycle tests

ID	Test	Expected result
MBX-01	Authenticate with correct mailbox key/challenge.	Access only to that mailbox ciphertext.
MBX-02	Replay/alter/expire mailbox challenge.	Rejected; no messages disclosed.
MBX-03	Reorder/delete/replace messages.	Sequence and previous-hash verification detects change.
AUTH-01	Assigned handler opens case using WebAuthn.	Allowed and signed access event recorded.
AUTH-02	Unassigned handler/admin attempts access.	Denied and attributable event recorded; no plaintext.
AUTH-03	Attempt invalid lifecycle transition or stale row version.	Rejected without partial event.
LIFE-01	Exercise acknowledgment/assignment/finding/extension/appeal timers.	SLA state and signed events match policy.
RED-01	Try to publish without complainant opt-in or reviewer approval.	Blocked.
RED-02	Seed names, NIC-like strings, GPS, faces, and document metadata in draft.	Automated checks flag; reviewer workflow prevents accidental release in test cases.
VOTE-01	Cast 9 valid votes.	Counts may display per policy, but official outcome is INCONCLUSIVE.
VOTE-02	Cast at least 10 votes with exactly/above/below 60% principal choice.	Boundary outcome matches specification.
VOTE-03	Community result conflicts with handler.	CONFLICTING and auditor review; no automatic close/delete.
VOTE-04	Auditor freezes public listing.	Public derivative hidden/frozen, private case/history retained, signed reason logged.

11. Performance, resilience, retention, and accessibility

ID	Test	Expected result
PERF-01	Load proof-session/submission path at declared prototype concurrency.	p95 target met for proof-only 2 MB scenario; queues bounded; no secret logged.
PERF-02	Prove both circuits on supported mid-range phone.	Published time/memory/battery measurements meet product threshold.
PERF-03	Verify receipt/inclusion on mid-range phone.	Local verification under one second excluding retrieval.
DOS-01	Send oversized/malformed CBOR, proofs, and slow requests.	Early bounded rejection; no memory/CPU exhaustion.
RES-01	Kill one DB/storage node after acceptance.	No accepted complaint/log entry lost.
RES-02	Backup and restore each zone independently.	Integrity checks pass; restored zone has no credentials/data from another zone.
RES-03	Rotate receipt/log/checkpoint/handler keys.	Old objects remain verifiable/decryptable as policy requires; new objects use new key ID.
RET-01	Advance clock through challenge/log/evidence retention schedules.	Expired transient rows and eligible evidence deleted; permanent commitments remain.
RET-02	Apply/release legal hold.	Timed deletion pauses/resumes with signed events.
A11Y-01	Keyboard, screen-reader, zoom, contrast, and focus audit.	WCAG 2.2 AA acceptance; cryptographic status is understandable without color alone.
I18N-01	Complete critical flows in Sinhala, Tamil, and English.	No untranslated security decisions, clipping, or ambiguous terminology.

12. Security tooling and review

- Property/fuzz tests for CBOR decoders, state machines, Merkle proofs, and API parsers.
- Static analysis and language linters on every change.
- Dependency, license, secret, container, and infrastructure-as-code scanning.
- Reproducible circuit/build verification in clean CI.
- Dynamic scanning of public and authenticated portals.
- Manual authorization, business-logic, privacy, mobile storage, and cryptographic review.
- External review of circuits, RSABSSA integration, recovery protocol, log/witness design, and anonymity claims.

Automated scanners are evidence, not substitutes for review.

13. Defect policy

- **Critical:** identity-to-complaint linkage, forged proof/receipt/checkpoint, plaintext exposure, entitlement/nullifier bypass, or unrecoverable accepted-data loss. Blocks all release.

- **High:** practical cross-case access, recovery takeover, witness-finality bypass, key exposure, or persistent network identifier joined to complaint. Blocks release.
- **Medium:** limited metadata leak, hard-to-exploit denial of service, incomplete audit attribution, or policy bypass requiring privileged access. Requires owner/date and explicit pilot approval.
- **Low:** defense-in-depth or documentation issue with no direct security-property violation. Track to closure.

14. Final acceptance report

The release report lists every test ID, result/evidence link, artifact and build IDs, open defects, measured performance, residual risks, external-review findings, operator/witness identities, key-ceremony records, and signatures of engineering, security, privacy, operations, accessibility, legal/governance, and product owners.